

AI WEATHERWISE

Intelligent Weather Forecasting and Insights

Team number: 11

Team Head: B.NIGITHA SREE

NM ID: (72E07787B690CE8B18A423F4DE4902C3)

Team Member: S.PRAVEENA

NM ID: (90F7D2ED683E3479A1C4B1AB5BDC31F9)

Team Member: R.RAMYASRI

NM ID: (04E2D103814009CA1CF4C81C4B4898E3)

Team Member: E.S.RISHMITHA

NM ID : (05E75793FCACAFBAC394AD3D13044BE4)

1. Project Overview

AI WeatherWise is a robust RESTful backend platform developed with Node.js and Express.js, using MongoDB with Mongoose ODM for persistent data management. It retrieves real-time weather information and integrates Google Gemini AI to generate natural-language summaries, personalized activity suggestions, and context-aware recommendations.

The platform provides JWT authentication, bcrypt password hashing, favorite-location management, centralized error handling, request sanitization, and a resilient fallback mode so core API functionality can remain available when external credentials are unavailable.

Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined service for tracking local weather conditions and receiving intelligent recommendations based on current environmental conditions.

Problem

- Difficulty tracking and managing multiple favorite weather locations.
- Limited personalization for outdoor activities and clothing choices.
- Raw weather metrics can be difficult to interpret quickly.
- Need for secure storage and synchronization of user preferences across devices.

Solution

AI WeatherWise provides a centralized backend that retrieves weather data, manages authenticated users and favorite locations, and orchestrates Gemini AI requests. The service converts live conditions into understandable summaries and practical recommendations.

Outcome

The platform reduces the effort required to interpret weather data while maintaining structured user data through Mongoose schemas. AI-generated insights add context to raw metrics and support practical weather-related decisions.

2. Requirements

Software Requirements

- Windows 10/11, macOS, or Linux.
- Node.js v16 or above and npm v8 or above.
- MongoDB with Mongoose ODM.
- Google Gemini SDK for AI integration.
- Postman or Thunder Client for REST API testing.
- Visual Studio Code or a similar IDE.

Hardware Requirements

- Intel Core i5 8th Gen or AMD Ryzen 5 equivalent or above.
- 8 GB RAM minimum; 16 GB recommended.
- At least 1 GB available storage.
- Internet connectivity for external weather and Gemini services.

3. Project Architecture

AI WeatherWise follows a modular MVC-oriented backend architecture. The Model layer defines persistent data structures, the Controller layer implements application workflows, and the API Routing layer serves as the interface for this headless backend.

Major Components

- API Interface — Postman and client applications send HTTP requests and receive JSON responses.
- Express Server Gateway — Handles HTTP methods, JSON parsing, CORS, ports, and route registration.
- Authentication and Authorization Middleware — Validates JWT tokens and protects guarded resources.
- Controller/Service Layer — Coordinates authentication, location management, weather retrieval, and AI insight generation.
- Database Interface — Mongoose translates application operations into MongoDB queries.
- AI Service Integration — Gemini generates weather summaries and personalized recommendations from live weather context.

MVC Responsibilities

- Model Layer — Defines User and Location schemas, validation rules, references, and persistence behavior.
- Controller Layer — Validates requests, executes business workflows, communicates with models/services, and returns JSON responses.
- View Layer — Since this is a headless API, the conventional visual view is represented by the API routing/interface layer.

4. Data Architecture and ER Model

User Entity

- `_id` — Primary key (MongoDB ObjectId).
- `name` — Required string.
- `email` — Required and unique string.
- `password` — Required hashed password.
- `role` — User role; default: reader.

Location Entity

- `_id` — Primary key (MongoDB ObjectId).
- `user` — Reference to the User collection.
- `city` — Required string.
- `country` — Required string.
- `createdAt` — Date with current-time default.

Relationship

A User has a one-to-many relationship with Location records. One registered user can save multiple favorite locations, while each location references its owning user.

Security and Data Controls

- JWT-based route protection.
- Role-based authorization for restricted operations.
- bcrypt password hashing.
- Payload sanitization against unsafe input and NoSQL manipulation risks.
- Centralized error handling for consistent responses.

5. Features and Roles

Key Features

- User registration and login.
- Secure JWT authentication.
- Favorite location CRUD operations.
- Real-time weather retrieval.
- Gemini-powered weather summaries and recommendations.
- Supported public weather endpoints.
- Fallback operation when external service credentials are unavailable.
- API health and integration verification.

Roles and Responsibilities

Admin

- Manage user accounts through permitted administrative operations.
- Monitor API health and system logs.
- Oversee database integrity and service availability.

Registered User

- Create and maintain favorite locations.
- Request current weather data.
- Request AI-generated summaries and recommendations.
- Manage supported authentication/profile information.

Public Weather User

Public clients can consume supported weather endpoints without an account, subject to the access rules implemented by the backend.

6. User Flow

1. User registers or logs in.
2. Server validates credentials and issues a JWT after successful authentication.
3. Authenticated user saves a favorite city.
4. User requests current weather for the selected city.
5. Backend retrieves and structures weather data.
6. User requests AI-driven insights.
7. Gemini generates a summary or recommendation when available.
8. Server validates authorization and returns the JSON response.

7. Project Setup and Configuration

Initialization

Create a folder named AI WeatherWise, open it in Visual Studio Code, and initialize the Node.js project.

```
npm init -y
```

Dependencies

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

```
npm install --save-dev nodemon
```

Core Files

- index.js
- .env
- package.json
- .gitignore

Environment Configuration

Keep secrets in the root .env file and exclude the file from source control.

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret
```

```
GEMINI_API_KEY=your_gemini_api_key
```

Recommended Modular Structure

Separate configuration, models, controllers, routes, middleware, services, and utility modules. The entry point should initialize the application and connect these modules without placing all business logic in one file.

8. Backend and Database Development

Application Entry Point — [index.js](#)

The index.js file loads environment variables, creates the Express application, configures middleware, establishes the MongoDB connection through Mongoose, registers API routes, and starts the server after successful initialization.

MongoDB Configuration

The database layer establishes communication between Node.js and MongoDB. Mongoose defines schemas, validates fields, manages references, and performs database operations.

Schema Management

The User schema stores identity, credentials, and role information. The Location schema stores favorite cities and references the corresponding User. Validation and default timestamps support consistent data.

Operational Flow

9. Load environment configuration.
10. Initialize Express and middleware.
11. Connect to MongoDB using Mongoose.
12. Register authentication, location, weather, and AI routes.
13. Start the server on the configured port.
14. Process requests through middleware, controllers, services, and models.
15. Return JSON responses or centralized error responses.

9. Execution and API Testing

Running the Backend

16. Ensure MongoDB is running and environment variables are configured.
17. Run npm install.
18. Start the development server with npm start.
19. Confirm successful database connection and server startup.

API Testing

Postman or Thunder Client can be used to verify status codes, authentication behavior, validation, JSON structure, database persistence, and failure handling.

Register API

Tests user registration, required fields, secure password processing, and creation of the User record.

Login API

Validates credentials and returns the authentication result, including the JWT required for protected operations.

Weather API

Retrieves current weather metrics for a requested city or supported location and returns structured JSON data.

Recommendation API

Uses weather context to request Gemini-generated summaries or recommendations. Configured fallback behavior can be used when the external AI service is unavailable.

Testing and Demonstration Assets

- API testing video: [API testing.mp4](#)
- Backend demonstration video: [Backend demo.mp4](#)
- Source code: [CodeDrive](#) / [Code](#)

10. Conclusion

AI WeatherWise demonstrates a secure and modular backend for intelligent weather services. By combining Node.js, Express.js, MongoDB, Mongoose, JWT authentication, and Gemini AI, the platform provides structured weather access, favorite-location management, and human-readable recommendations. Its MVC-oriented organization, centralized error handling, input sanitization, and fallback strategy improve maintainability, security, and resilience. The RESTful service can support web or mobile clients while keeping data management and AI orchestration centralized within the backend.